

Technical Manual

Bridge (Buildings
A-B) of the School
of Engineering,
UNAM



**Final Project
Created by:**

320249538

Group: 05

Semester: 2026-2

**Due date:
May 26, 2026**

Note on teamwork: This project was developed in collaboration with 423095203 during the course lab. This documentation specifically reflects my individual contributions, in accordance with the submission guidelines for the theory course. The sections on architecture, source code, and general scene description document the project in its entirety, explicitly indicating in each case which components were developed by me.

1. Project Objective

1.1 General objective

Develop and integrate a real-time, interactive 3D virtual environment that accurately reproduces the bridge connecting Buildings A and B on the first floor of the UNAM School of Engineering, applying the theoretical knowledge acquired during the course.

The environment simulates the typical layout of the bridge during academic fairs and recruitment events, incorporating six representative booths for sponsoring companies and student associations, as well as a complete system of dynamic lighting, animations, and free navigation via a synthetic camera.

1.2 Specific objectives

- Correct graphical implementation: Application of the Phong lighting model (ambient, diffuse, and specular components) to all loaded models, incorporating directional lights, point lights, and conical spotlights.
-
- 3D modeling: Construction of clean geometry with correctly oriented normals, consistent UV coordinates, and textures that reflect real materials (concrete, metal, glass, printed paper).
-
- Functional synthetic camera: Implementation of a first-person camera capable of freely exploring the scene using a keyboard and mouse.
-
- Seven animations: Each animation must have narrative context within the environment, visual coherence with the scene, and mathematical justification in the case of complex animations.
-
- Stable graphics system performance: Error-free execution, no detectable memory leaks, and acceptable performance on mid-range hardware.
-
- Project size optimization: Total models and textures under 100 MB, no compressed files in the repository, and no absolute paths in the source code.

1.2 Individual contributions

As part of this team project, my specific contributions were as follows:

3D Modeling:

Complete construction of the bridge model in Blender, including the deck, cylindrical support pillars, roof, perimeter railings, and side barriers.

Texturing of the bridge using UV mapping and the application of concrete and steel materials in Blender.

Lighting system:

Implementation of 24 spotlight-type lights distributed across the six stands (four per stand). Individual configuration of their position, direction, beam angles (interior and exterior), dimming, color, and intensity differentiated according to the active day/night mode.

Skybox system:

Design and implementation of the dual skybox system (interior + exterior) with four independent cubemaps (interior day, interior night, exterior day, exterior night), dynamic switching via the L key, and configuration of the corresponding shaders so that the interior skybox matches the bridge's actual dimensions derived from its geometry.

- **Animations (4 of 7):**

1. Walking visitor animation: Created using keyframes with linear step-by-step interpolation (10 keyframes). Implementation of dual-state logic to switch between the walking animation and a static pose of observation in front of the booths.
2. Booth presenter animation: Using a four-part hierarchy (body, head, forearm, hand) with rotations generated by independent sinusoidal functions for each joint.
3. Robotic arm animation: Using hierarchical keyframes with five levels of chained transformation (fixed base → pedestal → shoulder → elbow → claw) and nine keyframes describing a complete manipulation cycle.
4. Animation of the flying brochures: Combines trigonometric trajectories (sin/cos) with a two-phase logic (flight and rest) implemented using "fmod()", and a static stack of brochures piled on the table at the P&G booth.

2. Range and limitations

2.1 Scope of the project

- Free user movement: The user controls a first-person synthetic camera and can navigate to any point in the environment without predefined paths or movement restrictions.
- Switching between two lighting conditions: The user can dynamically switch between a daytime and a nighttime lighting scheme with a single keyboard command, observing how the behavior of the booth lights, ceiling lamps, and celestial bodies responds to the change.
- Selective animation activation: The user can individually enable or disable interactive animations using dedicated keys, allowing them to focus attention on specific elements of the environment during exploration.
- Local distribution and deployment: The project is delivered as a solution that can be compiled in Visual Studio, hosted in a public GitHub repository. Anyone can clone it, compile it, and run it on their own computer without additional configuration.

2.2 Limitations

- Collision detection was not implemented. The camera can pass through walls, pillars, stands, and any other models in the scene. The animated visitor also does not perceive obstacles and follows its predefined path regardless of what stands in its way.
- Dynamic shadows were not implemented. The Phong lighting model used does not include shadow mapping or ambient occlusion, so no object casts shadows on another. In night scenes, this limitation is noticeable, but correcting it would have required a more complex rendering pipeline.
- The camera operates exclusively in first-person view. No alternative views or a system for switching between multiple cameras were included. The choice of a single first-person camera aligns with the project's primary use case: simulating the perspective of a real visitor walking across the bridge.
- The total file size of the project was kept under 100 MB. This limit, mandated by the course requirements, constrained the maximum texture resolution and the polygon density of the models. Some elements could have benefited from higher-resolution textures, but the decision was made to prioritize strict compliance with the requirements over visual fidelity.

3. State of the art

3.1 Virtual campus systems

Virtual campus systems are graphical applications that use 3D geometry to replicate the physical facilities of an educational institution in order to facilitate navigation, event planning, or institutional outreach to prospective students.

One of the pioneering projects in this field was developed by Anhui Normal University in China, where a complete virtual campus system was built by combining SketchUp for 3D modeling and OpenGL for real-time rendering. The system proved to be executable on conventional PC hardware without specialized requirements, validating the viability of this type of academic application.

More recently, a project to model a university campus using Blender and Three.js explored the use of geometry scaled proportionally to the actual physical space, ensuring that the virtual model's dimensions accurately reflected the campus's architectural measurements.

In the web space, the University of Mauritius developed a comprehensive virtual tour of its campus using WebGL, a browser-optimized version of OpenGL. The explicit purpose was to help domestic and international students familiarize themselves with the campus before physically arriving. This use case is exactly the same as the one driving this project: to allow students, corporate sponsors, and external visitors to explore the bridge area and understand how booths are arranged during academic events, without the need for physical presence.

On a commercial level, companies such as THE FUTURE 3D offer professional virtual university campus tour services using Matterport technology and aerial photography with drones, capturing buildings, classrooms, halls, dormitories, and outdoor spaces in photorealistic detail.

3.2 Phong's lighting model

The lighting system implemented in this project is based on the reflection model proposed by Bui Tuong Phong in his 1975 doctoral thesis at the University of Utah. Phong's reflection model is an empirical local lighting model that calculates the appearance of a surface at a given point; it was considered radical at the time but has since become the standard shading method for numerous rendering applications.

Phong's model consists of three main components: ambient, diffuse, and specular lighting. This decomposition is exactly the one used in the "lighting.frag" shader of this project, where each fragment calculates its final color by summing the contributions of the three components for each light source present in the scene.

3.3 Graphics libraries used

OpenGL: A cross-platform graphics API for hardware-accelerated 2D and 3D rendering.

GLFW: A window and input event handling library optimized for OpenGL applications.

GLEW: An OpenGL extension loader that provides access to the API's modern features.

GLM (OpenGL Mathematics): A vector and matrix math library designed specifically for graphics programming, with GLSL-compatible syntax.

Assimp: A 3D model import library that supports over forty different formats, including OBJ and FBX.

SOIL2: A texture loading library optimized for OpenGL.

3.4 Overview of this project

Given the state of the art described above, this project lies at the intersection of three areas: virtual campus academic systems, interactive simulations for planning institutional events, and educational environments for teaching computer graphics. The features that set it apart are:

1. Specificity of the use case: Unlike general virtual tours, the project focuses on a specific area of the campus (the bridge between buildings A and B) and on a very specific use case (the simulation of fairs and events with booths), rather than an exhaustive representation of the entire campus.
2. Combination of animation techniques: The project simultaneously integrates skeletal animation, keyframe animation with linear interpolation, hierarchical animation, and animation using trigonometric functions.
3. Dual day/night lighting: Unlike most academic virtual tours, which use a single fixed lighting condition, the project incorporates a dynamic system that allows switching between two complete lighting schemes.

4. Technologies used and rationale

4.1 Language and graphics engine

C++ (Visual Studio 2022)

C++ was chosen as the primary language due to its direct access to system memory, its efficiency in computationally intensive operations, and its native compatibility with modern graphics libraries. Unlike higher-level languages such as Python or JavaScript, C++ allows for explicit control over GPU buffer allocation, uniform management in shaders, and resource deallocation, which is essential in real-time graphics applications where frame rate is critical.

Furthermore, this approach was used throughout the course.

OpenGL

OpenGL was selected as the graphics API for three main reasons:

1. Academic standard: It is the API consistently used in the course's lab sessions, ensuring that the team mastered its syntax and usage patterns.
2. Programmable pipeline: Unlike the fixed pipeline of previous versions, it requires explicit programming of vertex shaders and fragment shaders, which reinforces a true understanding of the rendering process.
3. Cross-platform: OpenGL has extensive official documentation and an active community, which facilitates troubleshooting during development.

4.2 Support libraries

GLFW

Responsible for window creation, OpenGL context management, and handling keyboard and mouse events. It was chosen over alternatives such as GLUT or SDL because GLFW is specifically designed for modern OpenGL and offers direct support for the callbacks used in the project's synthetic camera

GLEW

Allows OpenGL extensions to be dynamically loaded at runtime, avoiding the need to manually link each function. It is the standard solution for accessing modern API functions on Windows systems, where drivers do not directly expose all OpenGL functions.

GLM

A vector and matrix math library with syntax intentionally similar to GLSL. This similarity reduces friction when passing data between the CPU's C++ code and the GPU's shaders.

GLM was used in the project to construct transformation matrices (translation, rotation, scaling) for the hierarchical animations of the robotic arm and the presenter, to calculate the camera's view and projection matrices, and to apply rotations around pivots a technique used in all the hierarchical animations I developed.

Assimp

Used for loading 3D models in OBJ and FBX formats. It allows loading OBJ files with their respective .mtl files and associated textures, processes the bone hierarchies and animation channels of FBX models, and supports multiple different formats, enabling work with models downloaded from various sources (CGTrader, Free3D, Mixamo) without the need for manual conversion.

SOIL2

Loads textures in JPG and PNG formats directly into video memory. It was chosen over alternatives like stb_image due to its more concise API for loading cubemaps, an essential feature for the skybox system I developed. SOIL2 also automatically manages mipmapping, improving the filtering of textures viewed from a distance.

4.3 Modeling software

Blender 5.1

It was used to build the bridge's geometry from scratch, including the deck, cylindrical piers, roof, railings, and guardrails. It was used to apply UV mapping and materials with concrete and steel textures to the bridge. It was used to define and extract the rotation pivots used in hierarchical animations. Blender's ability to display the local coordinates of the 3D cursor was key to obtaining the exact points on which the rotations of the robotic arm and the presenter are applied. It was used to export the models in OBJ format with materials and normals, and in FBX format with the armatures and baked animations. The choice of Blender over commercial alternatives like Maya or 3ds Max is based on three factors: it is free and accessible to students, it has robust exporters for OBJ and FBX that are compatible with OpenGL, and it has a large community that facilitates the resolution of technical issues during modeling. Additionally, its learning curve is shorter compared to the others.

4.4 Collaboration and version control tool

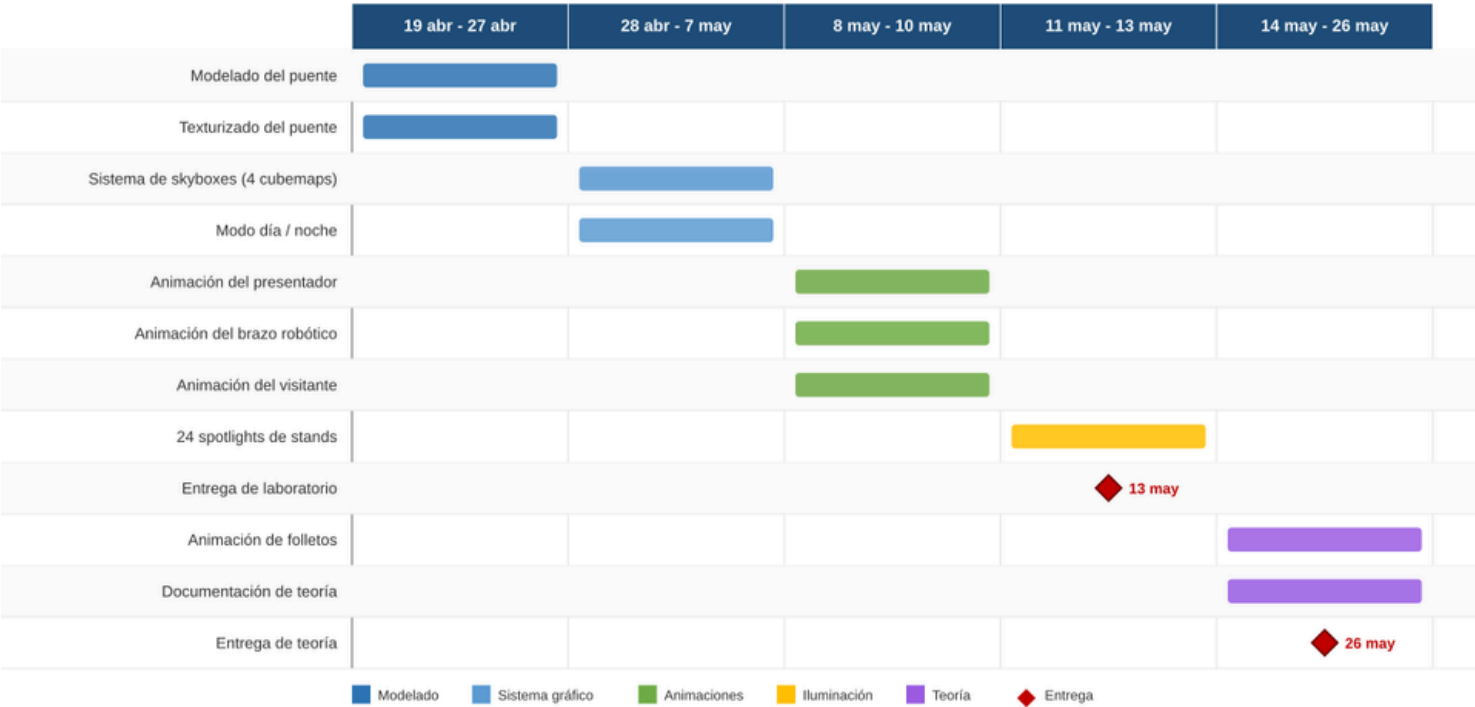
GitHub

It was used to maintain a complete history of changes to the project, allowing changes to be reverted in case of errors. Team members were able to work in parallel using separate branches, merging their changes once they were stable; it served as the project's delivery mechanism.

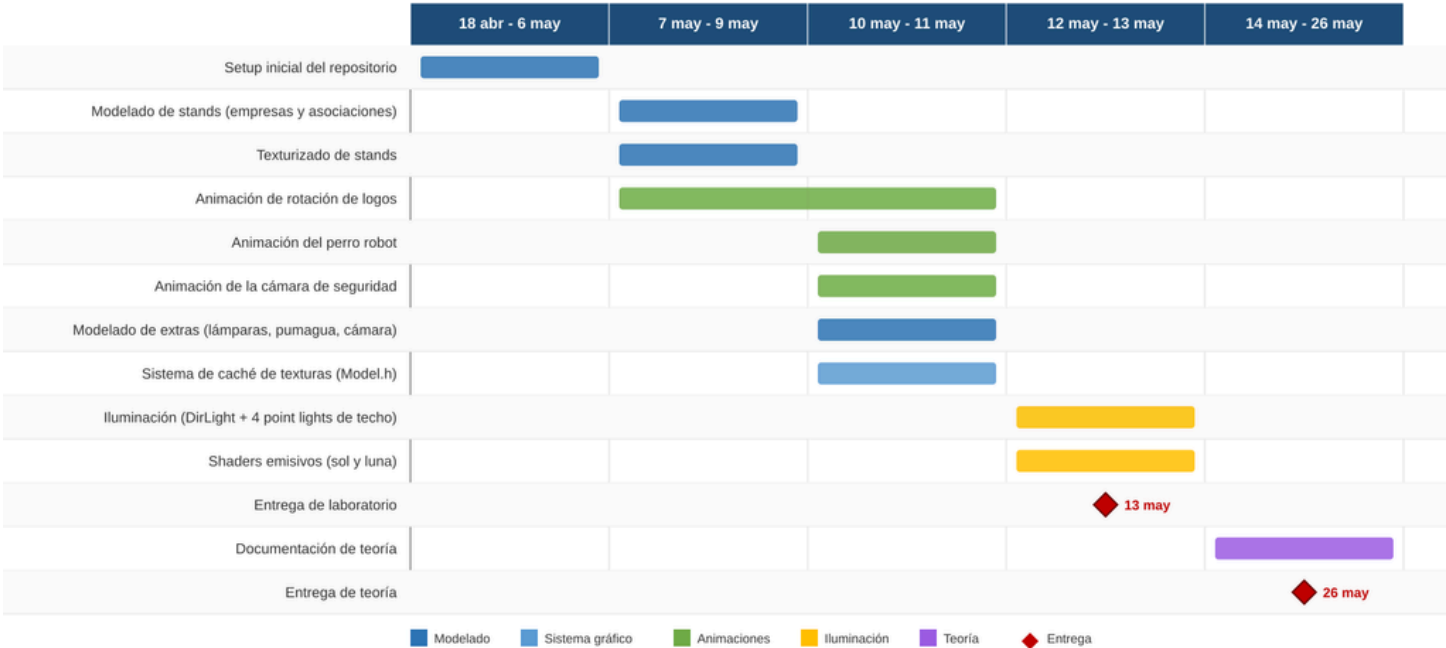
5. Gantt chart

This Gantt chart outlines my individual contribution to the project developed in collaboration with my colleague. The timeline is organized into five work periods spanning from the initial setup of the repository on April 19, 2026, to the final delivery of the theory documentation on May 26, 2026, and includes two deliverables: the lab project (May 13) and the theory project (May 26). The dates and task assignments are based on my actual commit history on GitHub under the username SamuelMV09, as evidenced by the information available in the project's public repository.

Gantt chart by 320249538

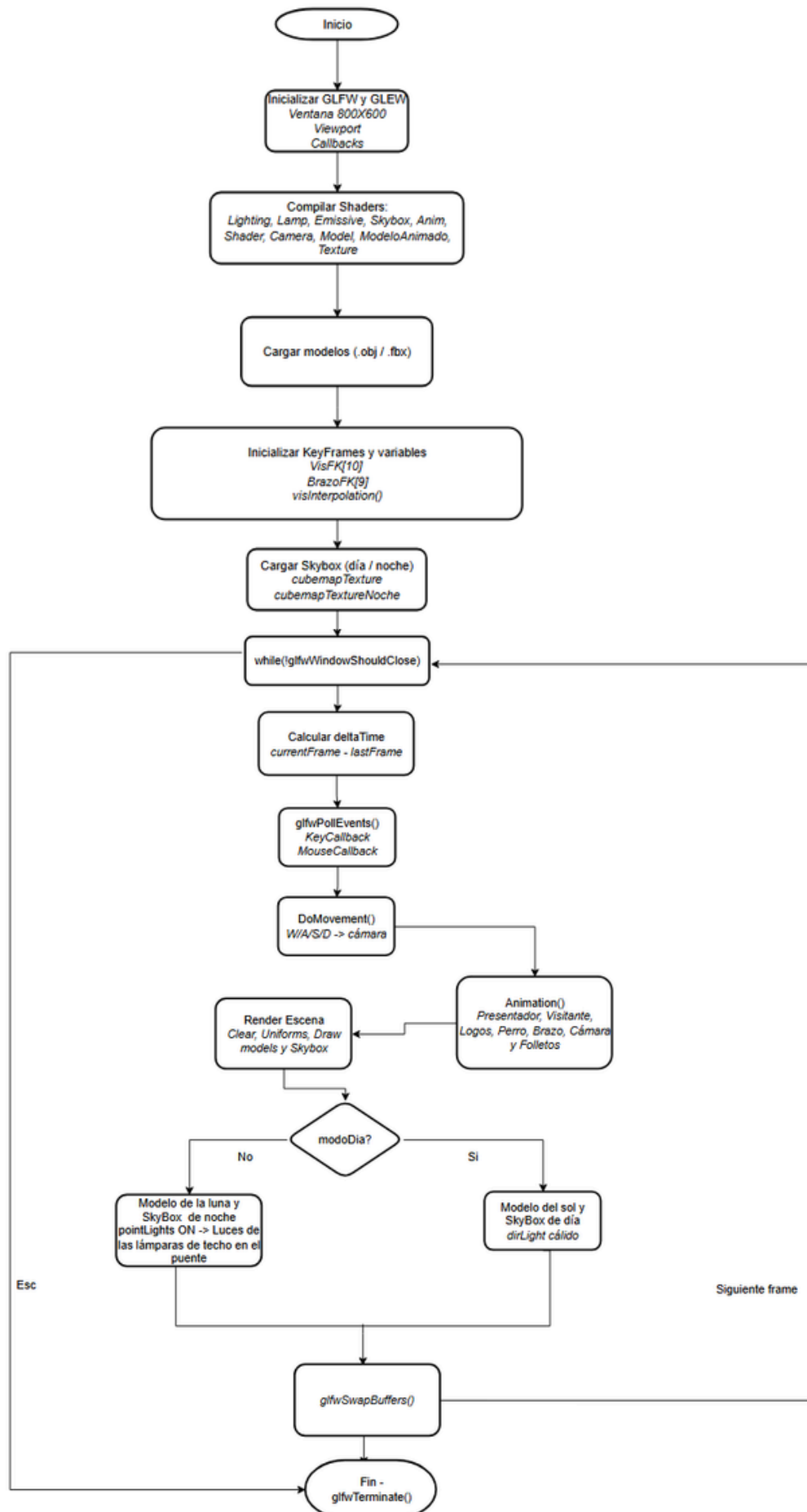


Gantt chart by 423095203

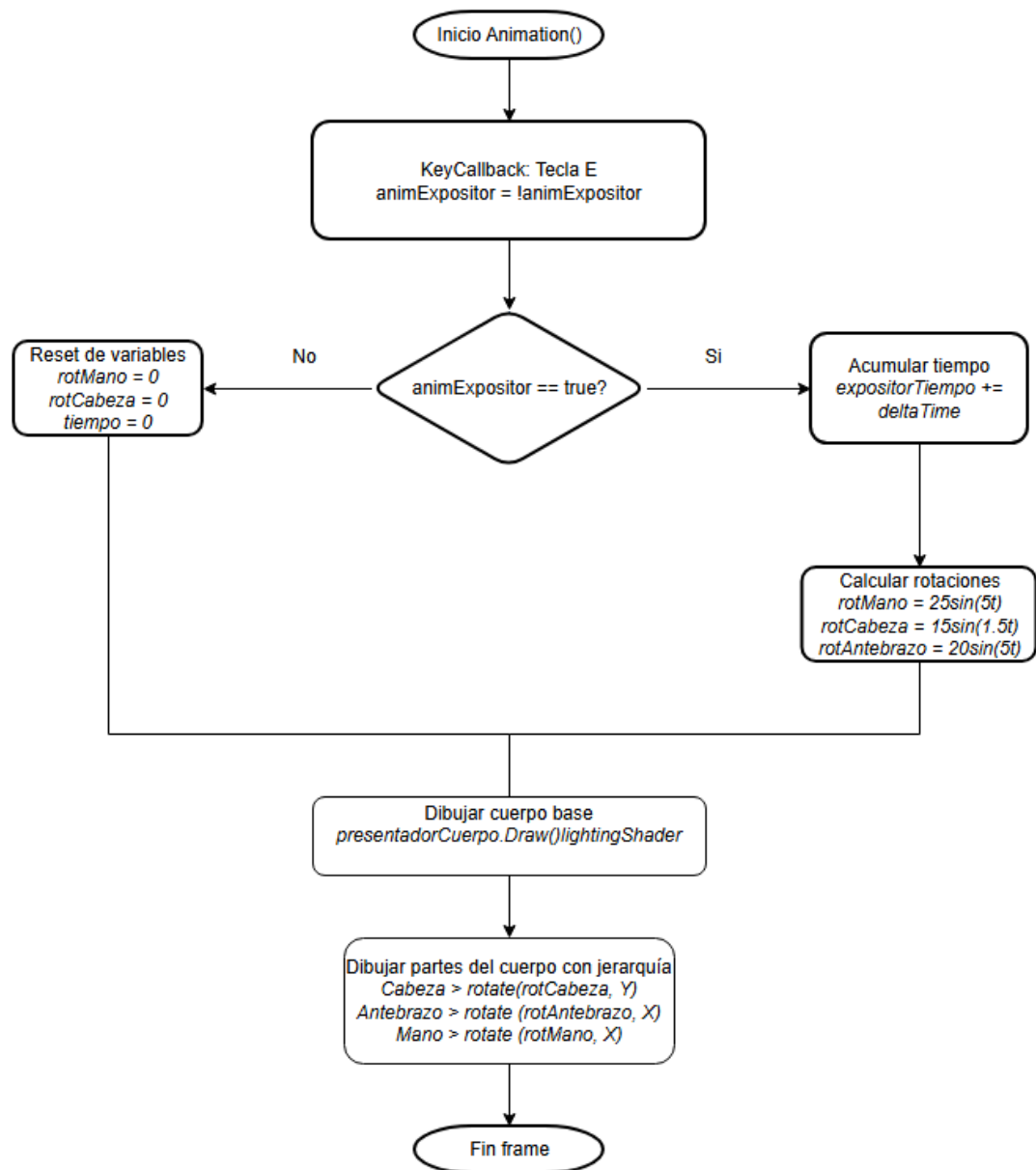


6. System flowcharts

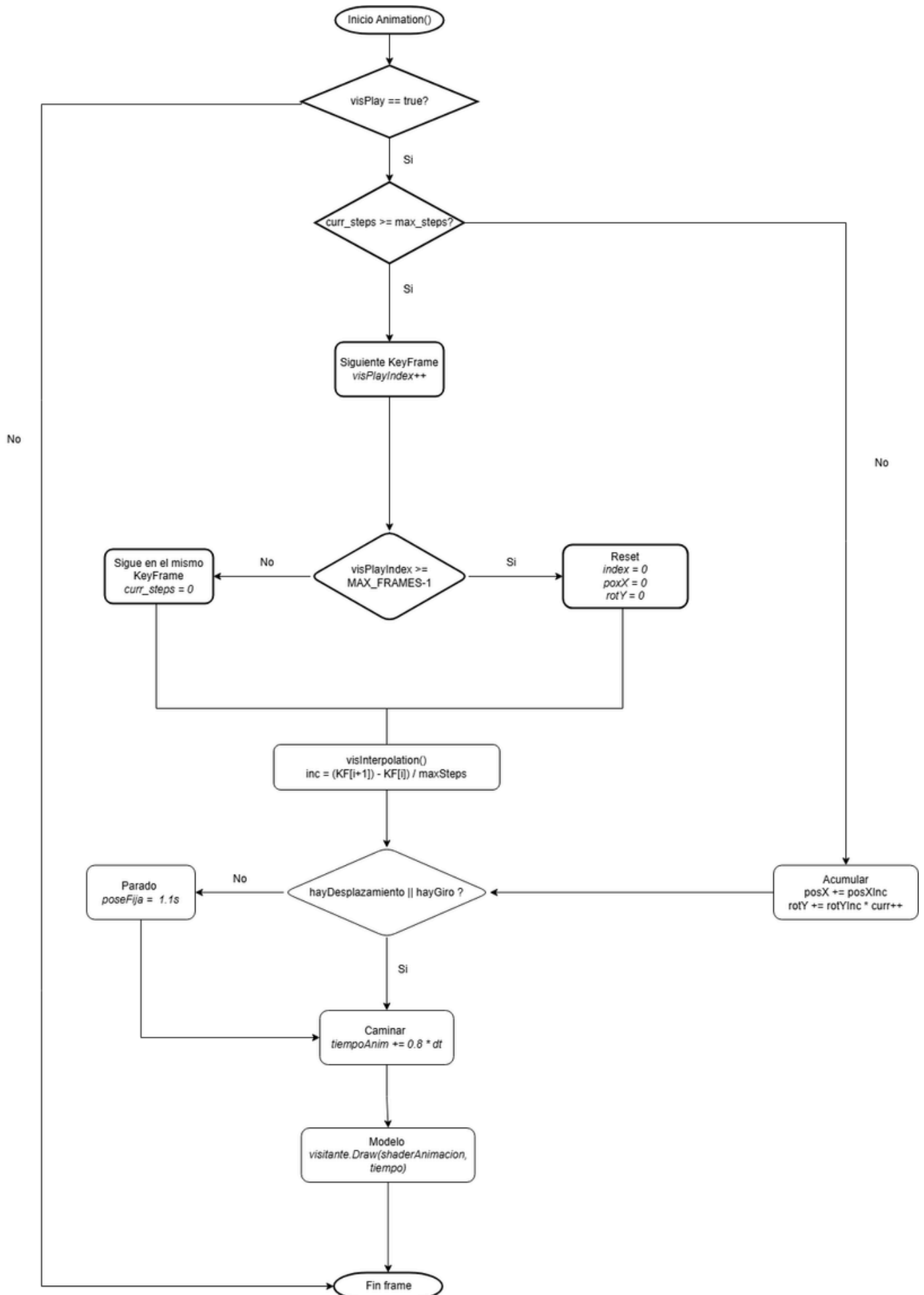
6.1 System Overview Diagram



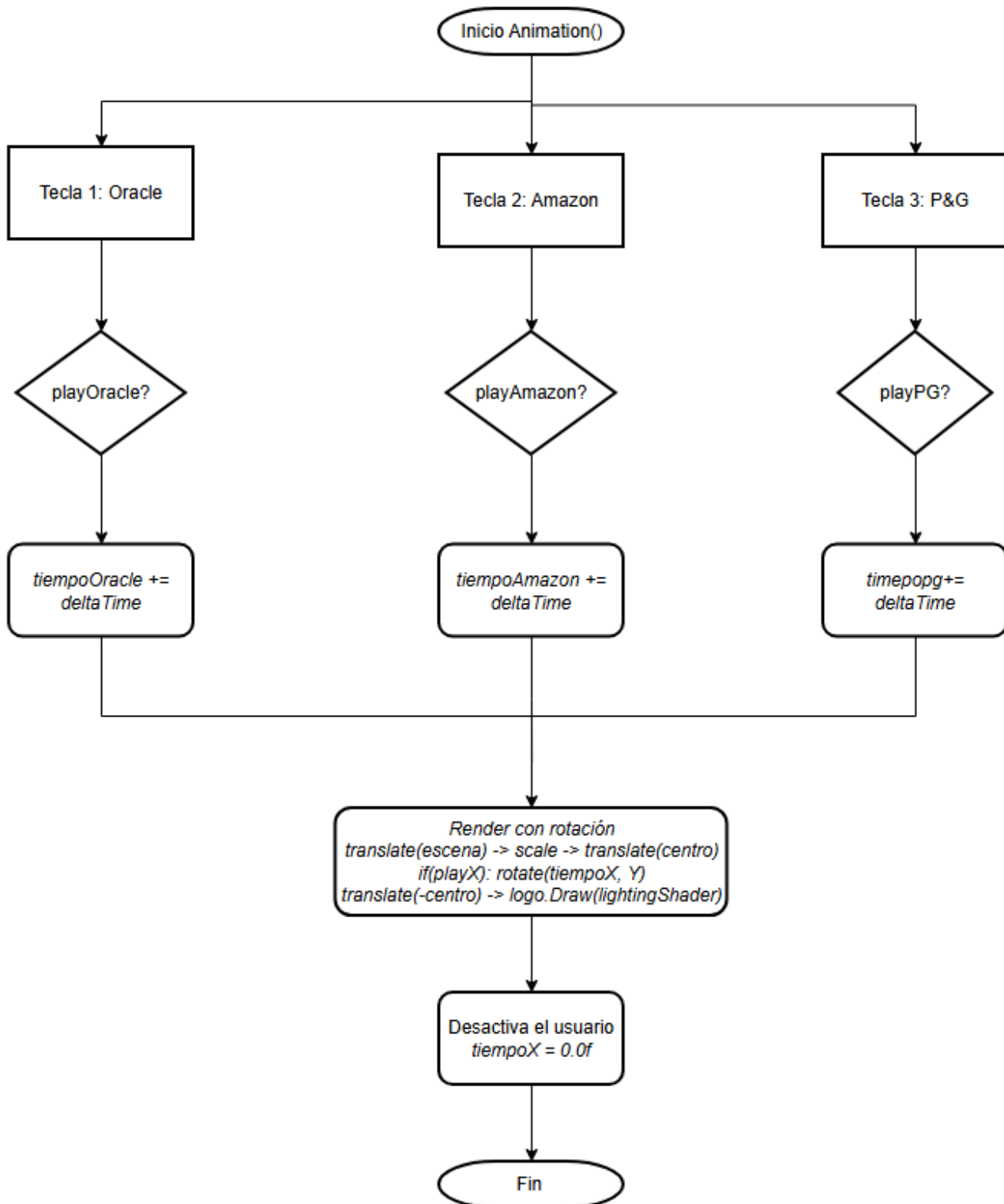
6.2 Diagram of the presenter's animation



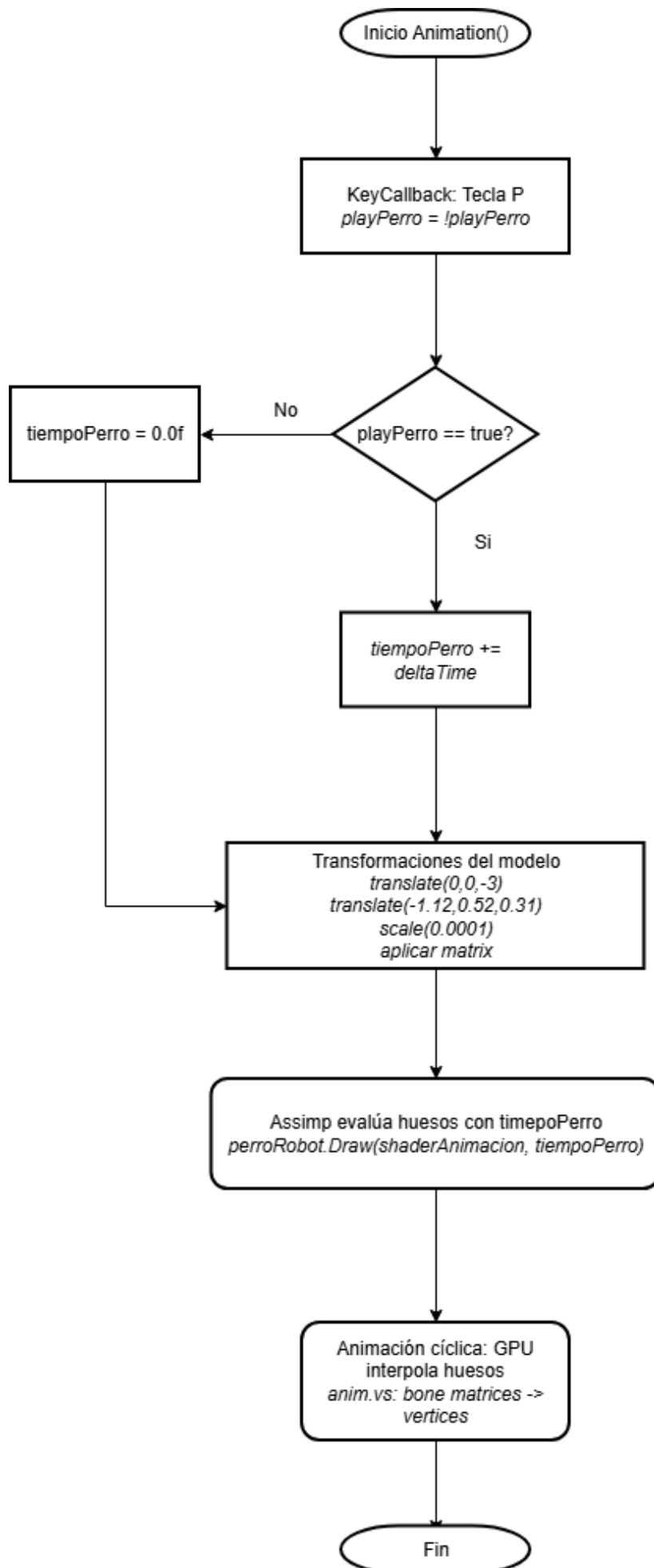
6.3 Diagram of the visitor animation



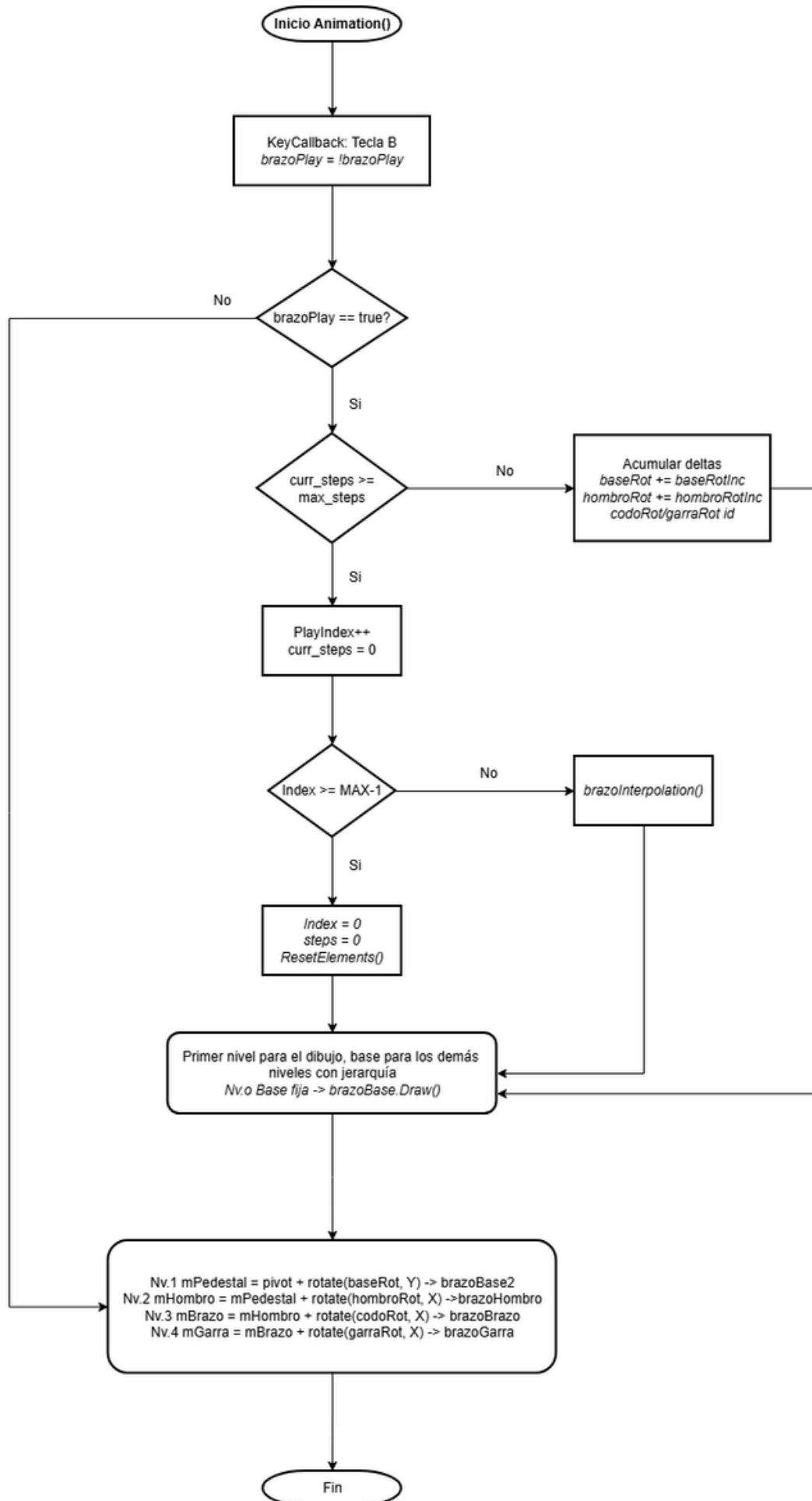
6.4 Diagram of the logo animation



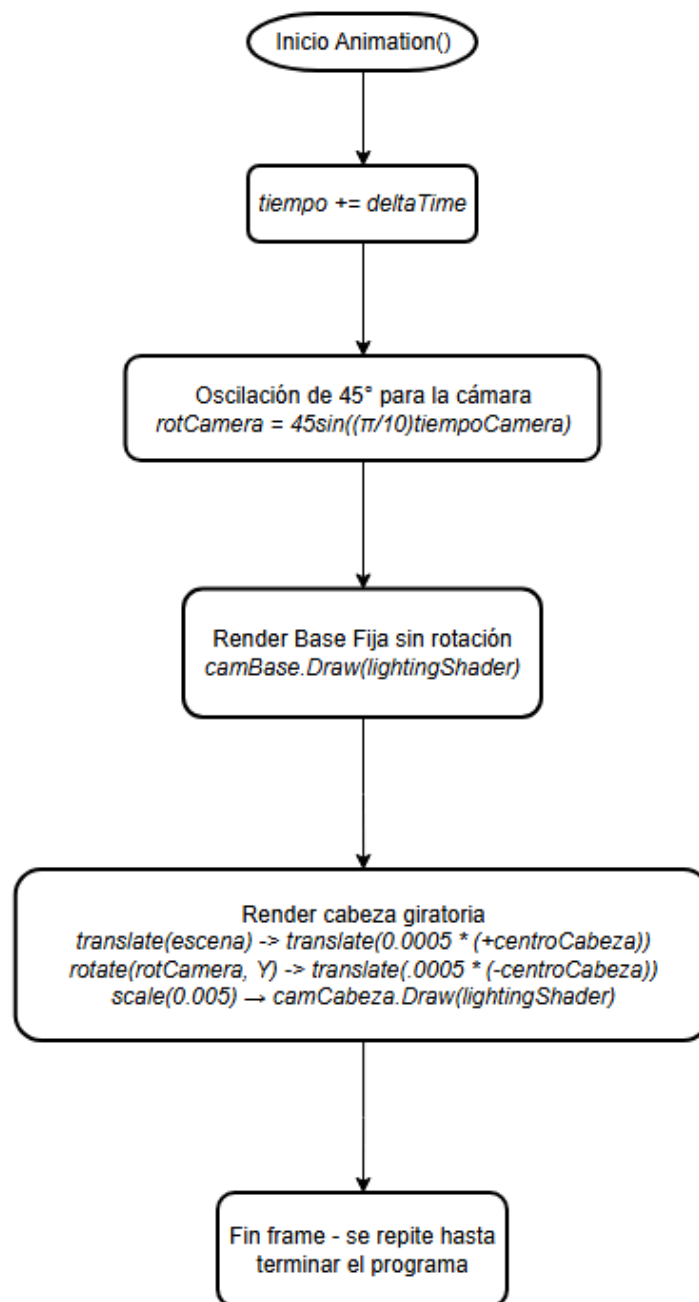
6.5 Diagram of the robot dog animation



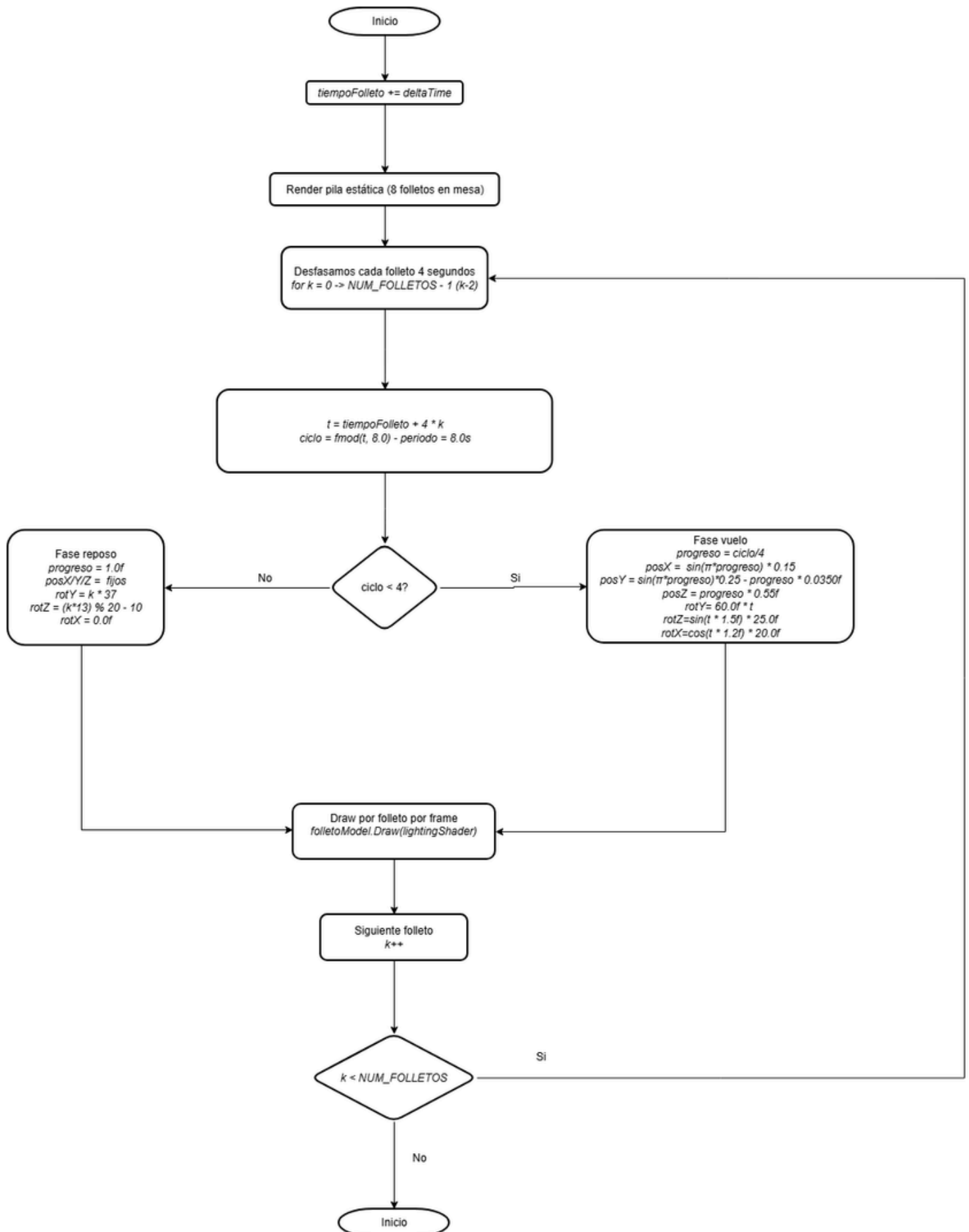
6.6 Diagram of the robotic arm animation



6.7 Diagram of the security camera animation



6.8 Diagram of the brochure animation



7. Code documentation

This section describes the most relevant technical components of the developed system, explaining the inner workings of the source code, the implemented algorithms, and the design decisions made during development. The documentation is organized into three main sections: system initialization, the Phong-based lighting system, and the dual skybox system with dynamic day/night switching. The following section documents in detail the seven independent animations that make up the project.

System Initialization

Objective

Before entering the main rendering cycle (game loop), the system performs a configuration sequence that sets up the OpenGL context, loads all visual resources for the environment, and defines the initial states of the animations.

OpenGL Context Configuration

Initialization begins with creating the window using GLFW and configuring the graphics context:

```
glfwInit();
GLFWwindow* window = glfwCreateWindow(WIDTH, HEIGHT, "ProyectoFinal-Samuel Medina Villa-Eric
Ramirez", nullptr, nullptr);
glfwMakeContextCurrent(window);
glewExperimental = GL_TRUE;
glfwInit();
glViewport(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT);
```

Once the context has been initialized, keyboard and mouse callbacks are registered, allowing the user to interact with the synthetic camera and trigger animations using dedicated keys.

Loading Models and Shaders

After the context is set up, the system loads all the 3D models and shader programs required for rendering. This loading occurs only once at the start of the program, avoiding reloads during the rendering cycle:

```
Shader lightingShader("Shader/lighting.vs", "Shader/lighting.frag");
Shader lampShader("Shader/lamp.vs", "Shader/lamp.frag");
Shader emissiveShader("Shader/emissive.vs", "Shader/emissive.frag"); //sol y luna
Shader skyboxshader("Shader/skybox.vs", "Shader/skybox.frag");
Shader shaderAnimacion("Shader/anim.vs", "Shader/anim.frag");
```

```

Model Puente((char*)"Models/Puente/Puente.obj");
Model mamp1((char*)"Models/stands_2/mamp_1.obj"); .....

//stands de empresas
Model stand_pag((char*)"Models/stands_1/pg.obj");
Model stand_amazon((char*)"Models/stands_1/amazon.obj");
Model stand_oracle((char*)"Models/stands_1/oracle.obj");

//stands de asociaciones
Model stand_crofi((char*)"Models/stands_1/crofi.obj"); .....

```

Initializing Keyframes

Keyframe-based animations (visitor and robotic arm) require their keyframe sequences to be defined before entering the game loop. This initialization is performed using the `resetElements()` function and the first call to `interpolation()`, which calculates the step increments between the first and second keyframes:

```

visResetElements();
visPlay = true;
visPlayIndex = 0;
vis_i_curr_steps = 0;
visInterpolation();

```

Once initialization is complete, the system enters the main rendering loop, which runs continuously until the user closes the window.

Lighting System

Objective

It provides visual realism to the environment through an approximate physical simulation of how light behaves on model surfaces, and allows for the creation of two completely different atmospheres—day and night—via a dynamic, real-time transition.

The system implements the Phong reflection model, calculated per fragment on all static models in the environment using the `lighting.frag` shader. The model simultaneously combines a global directional light, four point lights, and twenty-four conical reflectors

Directional Light (Sun/Moon)

Used to represent the Sun during the day and the Moon at night. Its behavior is modified in real time based on the value of the `dayMode` flag:

```

if (modoDia) {
    //SQL
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.direction"), 1.3f, -1.1f, 1.0f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.ambient"), 0.35f, 0.32f, 0.28f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.diffuse"), 0.80f, 0.76f, 0.68f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.specular"), 0.45f, 0.45f, 0.40f);
}
else {
    //LUNA
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.direction"), -1.3f, -1.1f, 1.0f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.ambient"), 0.08f, 0.10f, 0.18f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.diffuse"), 0.20f, 0.25f, 0.40f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.specular"), 0.30f, 0.32f, 0.45f);
}

```

Spotlights on the stands (24 spotlights)

Each of the bridge's six stands has four spotlight-type lamps that cast a focused beam on their promotional elements. The total of twenty-four conical spotlights is managed via an array in the fragment shader (standSpots[0..23]), whose positions, directions, and properties are configured individually.

Each spotlight is configured within a for loop that assigns its uniform parameters based on the stand to which it belongs; for practical purposes, the example of a stand is provided.

```

for (int i = 0; i < 4; i++) {
    std::string base = "standSpots[" + std::to_string(i) + "]";

    //posicion
    glUniform3f(glGetUniformLocation(lightningShader.Program, (base + ".position").c_str()),
        oracleSpots[i].x, oracleSpots[i].y, oracleSpots[i].z);
    glUniform3f(glGetUniformLocation(lightningShader.Program, (base + ".direction").c_str()),
        0.0f, -1.0f, 0.0f);

    //angulos de apertura
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base + ".cutOff").c_str()),
        glm::cos(glm::radians(8.0f)));
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base + ".outerCutOff").c_str()),
        glm::cos(glm::radians(30.0f)));

    //color de la luz: blanco
    glUniform3f(glGetUniformLocation(lightningShader.Program, (base + ".ambient").c_str()),
        0.0f, 0.0f, 0.0f); //sin ambiente para evitar que ilumine fuera del cono y dar efecto exagerado del
spotlight
    glUniform3f(glGetUniformLocation(lightningShader.Program, (base + ".diffuse").c_str()),
        1.0f * spot_intensity, 1.0f * spot_intensity, 1.0f * spot_intensity);
    glUniform3f(glGetUniformLocation(lightningShader.Program, (base + ".specular").c_str()),
        1.0f * spot_intensity, 1.0f * spot_intensity, 1.0f * spot_intensity);

    //atenuacion
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base + ".constant").c_str()), 1.0f);
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base + ".linear").c_str()), 5.0f);
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base + ".quadratic").c_str()), 50.0f);
}

```

Point lights on the ceiling (4 point lights)

The four lamps distributed along the bridge ceiling are modeled as point lights that are activated only during night mode. These lights use a dimming table calibrated for a distance of approximately 100 units, with specific values for the constant, linear, and quadratic coefficients:

```
glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[0].constant"), 1.0f);
glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[0].linear"), 0.045f); //usamos
los valores de linear y quadratic para distance 100
//dados en la tabla de pointLights vista en clase de teoria (grupo 05)
glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[0].quadratic"), 0.0075f);
```

Skybox System

Objective

The skybox system provides the visual environment surrounding the user. A dual system was implemented, consisting of two simultaneous skyboxes: an interior one that fits the bridge's volume and an exterior one that surrounds the entire bridge structure. Each has two versions—day and night—which can be dynamically switched using the L key.

Storage Structure

The project manages four independent cubemaps, each consisting of six textures representing the faces of a cube (right, left, top, bottom, front, back). The textures are loaded into video memory using the LoadCubemap function of the TextureLoading utility class:

```
GLuint cubemapTexture          = TextureLoading::LoadCubemap(faces);
GLuint cubemapTextureExterior  = TextureLoading::LoadCubemap(facesExterior);
GLuint cubemapTextureNoche     = TextureLoading::LoadCubemap(facesNoche);
GLuint cubemapTextureExteriorNoche =TextureLoading::LoadCubemap(facesExteriorNoche);
```

Outside skybox

The outside skybox is rendered at the start of the game loop, before any opaque models. The depth mask is disabled during its rendering, which prevents the skybox from blocking the visibility of models behind it:

```
glDepthFunc(GL_LEQUAL);
glDepthMask(GL_FALSE);
skyboxshader.Use();
// ... configuración de uniformes ...
glDrawArrays(GL_TRIANGLES, 0, 36);
glDepthMask(GL_TRUE);
glDepthFunc(GL_LESS);
```

Inside skybox

Unlike the outside skybox, the inside skybox must fit the bridge's volume exactly so that the sky visible through the side openings matches the actual perspective of the surroundings. To achieve this, a custom model matrix is applied that positions and scales the inside skybox cube according to the bridge's dimensions, obtained directly from the model's vertex data in Blender:

```
model = glm::mat4(1);  
model = glm::translate(model, glm::vec3(-0.0413f, 0.6800f, -2.9475f));  
model = glm::scale(model, glm::vec3(1.4619f, 0.1850f, 0.4875f));
```

Dynamic Day/Night Switch

The switch between day mode and night mode is controlled by the L key, which toggles the value of the `modeDia` boolean flag. In each iteration of the game loop, this variable determines which cubemap is loaded when the draw call is made, using a ternary condition:

```
glBindTexture(GL_TEXTURE_CUBE_MAP, modoDia ? cubemapTexture : cubemapTextureNoche);
```

The same pattern applies to the exterior skybox and its various versions.

Security Camera Animation

Objective

A security camera is mounted on one of the bridge's pillars; it consists of two separate models: a fixed base anchored to the pillar and a rotating head. The animation replicates the automatic panning motion characteristic of surveillance systems, creating a continuous oscillating movement of the head while the base remains stationary.

The objective of separating the model into two parts is to allow the rotation to be applied only to the rotating component, keeping the base attached to the pillar intact. This separation was performed beforehand in Blender by exporting the two components as separate OBJ models.

Mathematical basis

The panning motion is modeled using a continuous sinusoidal function applied to the head's rotation angle:

$$\text{rotCamera}(t) = 45^\circ \cdot \sin(t \cdot \pi/10)$$

where t represents the elapsed time since the start of the program, expressed in seconds. The sine wave function produces a natural oscillating motion with the following properties:

Amplitude: $\pm 45^\circ$ from the center, resulting in a total pan range of 90° .

Angular frequency: $\pi/10$ rad/s.

Period: 20 seconds per complete cycle (round trip).

Implementation

The angle is updated in each iteration of the game loop within the Animation() function:

```
tiempoCamera += deltaTime;  
rotCamera = 45.0f * sin(tiempoCamera * (glm::pi<float>() / 10.0f));
```

The rotation is applied exclusively to the camCabeza model, using the physical center of the head as the pivot point

```
modelCamCabeza = glm::translate(modelCamCabeza, centroCamCabeza * 0.005f);  
modelCamCabeza = glm::rotate(modelCamCabeza, glm::radians(rotCamera), glm::vec3(0.0f, 1.0f, 0.0f));  
modelCamCabeza = glm::translate(modelCamCabeza, -centroCamCabeza * 0.005f);  
modelCamCabeza = glm::scale(modelCamCabeza, glm::vec3(0.005f, 0.005f, 0.005f));
```

Playback Controls

This animation plays automatically when the program starts and remains active indefinitely throughout the application's lifecycle. It does not require user interaction and cannot be paused, as it replicates the expected behavior of a continuously operating surveillance system.

Logo Rotation Animation

Objective

A separate three-dimensional logo floats above each of the three sponsor company booths (Oracle, Amazon, and P&G) and can rotate around its vertical axis. The animation implements a cumulative procedural rotation that allows the user to enable or disable each logo independently using the number keys, creating an attractive visual effect similar to that of real promotional displays.

Technical Basis

The rotation is constructed using an incremental transformation applied to each frame on the model matrix of each logo. The angular velocity is constant, and the accumulated angle is stored in a separate time accumulator for each logo:

```
if (playOracle) tiempoOracle += deltaTime;  
if (playAmazon) tiempoAmazon += deltaTime;  
if (playPG)      tiempoPG      += deltaTime;
```

The transformation applied uses the physical center of the logo as a pivot point, extracted from the scene in Blender, ensuring that the rotation occurs around the model's local vertical axis rather than the global coordinate origin

Example using the Oracle booth; the process is the same for the other two logos

```
glm::vec3 centroOracle(-255.8f, 136.83f, -48.15f);

modelOracle = glm::translate(modelOracle, centroOracle);
if (playOracle)
    modelOracle = glm::rotate(modelOracle,
        tiempoOracle, glm::vec3(0.0f, 1.0f, 0.0f));
modelOracle = glm::translate(modelOracle, -centroOracle);
```

Playback Control

Each logo has its own independent activation flag and is controlled by a dedicated numeric key:

Key 1: Enables or disables rotation of the Oracle logo.

Key 2: Enables or disables rotation of the Amazon logo.

Key 3: Enables or disables rotation of the P&G logo

When any rotation is disabled, the corresponding timer is reset to zero, which returns the logo to its original orientation

```
if (key == GLFW_KEY_1 && action == GLFW_PRESS) {
    playOracle = !playOracle;
    if (!playOracle) tiempoOracle = 0.0f;
}
```

Animation of Flying Flyers

Objective

Simulate the typical behavior of promotional flyers at an academic event: a stack of flyers rests on the table at the P&G booth, and every so often a couple of flyers fly from the stack onto the bridge floor, where they eventually come to rest.

This animation is the most complex in the project from a mathematical standpoint because it combines three elements: a curved trajectory in three-dimensional space generated using trigonometric functions, a two-phase logic (flight and rest) controlled by a modulo function, and animated rotations during flight that freeze upon landing.

Mathematical basis

Each leaflet follows an eight-second cycle consisting of two phases. The time progression within the cycle is calculated using the modulo function:

$$\text{cycle} = \text{fmod}(t + k \cdot 4, 8)$$

where t is the total duration of the animation, k is the leaflet index (0 or 1), and the sum of $k \times 4$ creates a four-second delay between the leaflets, preventing them from flying in sync. The normalized progress value within each phase is calculated as:

$$\begin{aligned} \text{progress} &= \text{cycle} / 4 && (\text{if cycle} < 4, \text{flight phase}) \\ \text{progress} &= 1 && (\text{if cycle} \geq 4, \text{rest phase}) \end{aligned}$$

The leaflet's trajectory during the flight phase is modeled by combining sinusoidal functions and linear displacement

```
float posX = pilaPosX + sin(progreso * 3.1416f) * 0.15f;
float posY = pilaPosY + sin(progreso * 3.1416f) * 0.25f - progreso * 0.0350f;
float posZ = pilaPosZ + progreso * 0.55f;
```

Rotations during flight are also calculated using separate trigonometric functions for each axis:

```
rotY = t * 60.0f;
rotZ = sin(t * 1.5f) * 25.0f;
rotX = cos(t * 1.2f) * 20.0f;
```

Two-Phase Logic

The behavior of the leaflet changes radically as it transitions from the flight phase to the resting phase. This logic is implemented using a Boolean flag `enReposo`, which alters both the trajectory and the rotations:

```
float ciclo = fmod(t, 8.0f);
float progreso;
bool enReposo = false;

if (ciclo < fasevuelo) {
    progreso = ciclo / fasevuelo;
} else {
    progreso = 1.0f;
    enReposo = true;
}
```

During the resting phase, the leaflets' rotations freeze in a final position, simulating that the leaflet has fallen to the floor in a natural orientation. Each leaflet falls with a slightly different orientation calculated based on its index, which prevents both leaflets from ending up in exactly the same position:

```
if (enReposo) {
    rotY = (float)(k * 37);
    rotZ = (float)((k * 13) % 20 - 10);
    rotX = 0.0f;
} else {
    rotY = t * 60.0f;
    rotZ = sin(t * 1.5f) * 25.0f;
    rotX = cos(t * 1.2f) * 20.0f;
}
```

Static stack of brochures

To complement the animation of the flying brochures, a static stack of brochures is rendered on the table at the P&G booth, with slight variations in position and rotation, creating a natural appearance of piled-up papers:

```
for (int k = 0; k < 8; k++) {
    glm::mat4 modelPila = glm::mat4(1.0f);
    modelPila = glm::translate(modelPila, glm::vec3(
        pilaPosX + (k % 3) * 0.015f - 0.015f,
        pilaPosY + k * 0.001f,
        pilaPosZ + ((k * 7) % 5) * 0.008f - 0.02f));
    modelPila = glm::rotate(modelPila,
        glm::radians((float)(k * 13)), glm::vec3(0.0f, 1.0f, 0.0f));
    folletoModel.Draw(lightningShader);
}
```

The expressions $(k \% 3)$ and $(k * 7) \% 5$ generate pseudo-random patterns without the need for actual random number generators

Playback Control

The animation runs automatically when the program starts. The F key resets the timeFolleto counter, which is useful for repositioning the leaflets to their starting points if you want to observe the beginning of the flight again:

```
if (key == GLFW_KEY_F && action == GLFW_PRESS) {
    tiempoFolleto = 0.0f;
}
```

Visitor Walking Animation

Objective

The visitor is an animated character that continuously traverses the entire bridge. The animation combines two distinct techniques: keyframe animation with linear interpolation to control the character's overall movement and rotation, and skeletal animation using bones to reproduce the walking cycle on the FBX model loaded from Mixamo.

The goal of combining both techniques is to create a character that moves through space along a predefined path (controlled by keyframes) while performing a realistic walking cycle. This combination replicates the natural behavior of a visitor walking while observing the stands on the bridge.

Storage Structure

The keyframes of the path are stored in an array of VIS_FRAME structures. Each structure defines a spatial pose of the visitor at a given moment, along with the number of interpolation steps separating that keyframe from the next:

```
typedef struct _vis_frame {
    float visPosX;    float visPosXInc;
    float visRotY;    float visRotYInc;
    int maxSteps;
} VIS_FRAME;
```

Defining the route

Ten keyframes (VIS_MAX_FRAMES = 10) were defined to describe the visitor's complete cycle across the bridge. The route follows this sequence:

Start at the center and walk toward the right end of the bridge.

Turn 90° to look at one of the booths on the right side.

After a pause, turns another 90° and walks toward the opposite end.

Repeats the pause-and-turn pattern in front of a booth on the left side.

Walks back to the center and restarts the cycle indefinitely

The specific values of the keyframes are defined during program initialization:

```
VisKF[0] = { 0.0f, 0, 0.0f, 0, 800 };
VisKF[1] = { 240.0f, 0, 0.0f, 0, 50 };
VisKF[2] = { 240.0f, 0, 90.0f, 0, 200 };
VisKF[3] = { 240.0f, 0, 90.0f, 0, 50 };
VisKF[4] = { 240.0f, 0, 180.0f, 0, 1600 };
// ... continúa hasta el keyframe 9
```

Linear Interpolation

To create smooth motion between keyframes, linear interpolation is applied by calculating the increments per step. The general equation used is:

$$\text{Increment} = (\text{FinalValue} - \text{InitialValue}) / \text{NumberOfSteps}$$

The visInterpolation() function automatically calculates these increments each time a keyframe is completed and the animation moves to the next one:

```
VisKF[visPlayIndex].visPosXInc =(VisKF[visPlayIndex + 1].visPosX - VisKF[visPlayIndex].visPosX)/
vis_i_max_steps;
```

Dual-State Logic

As the visitor moves or turns, the walking cycle plays back using a timer (visitorAnimTime). When the visitor is standing still in front of a booth, the model freezes in a fixed pose that corresponds to the exact frame where the character appears to be standing and looking:

```

bool hayDesplazamiento = (std::abs(VisKF[visPlayIndex].visPosXInc) > 0.0001f);
bool hayGiro = (std::abs(VisKF[visPlayIndex].visRotYInc) > 0.0001f);

if (hayDesplazamiento || hayGiro)
{
    tiempoAnimVisitante += deltaTime * 0.8f;
    visitante.Draw(shaderAnimacion, tiempoAnimVisitante);
}
else
{
    visitante.Draw(shaderAnimacion, poseFijaVisitante);
}

```

Animation of the Stand Host

Objective

At one of the booths, there is an articulated character called the presenter, whose role within the environment is to welcome visitors who approach the area. The animation is constructed using a four-part hierarchy (body, head, forearm, and hand), whose rotations are generated from sinusoidal functions with different frequencies and amplitudes for each joint.

Mathematical basis

The rotation of each joint is calculated based on the elapsed time (exhibitorTime), applying the general equation:

$$\text{rotation}(t) = \text{amplitude} \cdot \sin(t \cdot \text{frequency})$$

where t is the elapsed time in seconds since the start of the animation, the amplitude determines the angular range of motion expressed in degrees, and the frequency determines the angular velocity in radians per second. The specific parameters assigned to each joint are as follows:

Forearm: amplitude 20° and frequency 5.0 rad/s , about the X-axis.

Hand: amplitude 25° and frequency 5.0 rad/s , about the Y-axis.

Head: amplitude 15° and frequency 1.5 rad/s , about the Y-axis.

Implementation

Rotations are calculated in each iteration of the game loop within the Animation() function whenever the animExpositor flag is active:

```

if (animExpositor)
{
    expositorTiempo += deltaTime;
    expositorRotMano = 25.0f * sin(expositorTiempo * 5.0f);
    expositorRotCabeza = 15.0f * sin(expositorTiempo * 1.5f);
    expositorRotAntebrazo = 20.0f * sin(expositorTiempo * 5.0f);
}

```

The calculated rotations are then applied to the transformation matrices of each body part, respecting the parent-child hierarchy: the body is the root node, the head and forearm are direct children of the body, and the hand is a child of the forearm. This allows the hand to automatically inherit the transformations of the forearm, reproducing the correct anatomical behavior:

```
glm::mat4 antebrazoMat = modelPres;
antebrazoMat = glm::translate(antebrazoMat, glm::vec3(-16.23f, 124.96f, -13.10f));
antebrazoMat=glm::rotate(antebrazoMat,glm::radians(expositorRotAntebrazo), glm::vec3(1.0f, 0.0f, 0.0f));
antebrazoMat = glm::translate(antebrazoMat, glm::vec3(16.23f, -124.96f, 13.10f));

glm::mat4 manoMat = antebrazoMat;
// La mano hereda la transformación del antebrazo
```

The rotation pivots were extracted directly from the model in Blender.

Playback Controls

The animation is toggled on or off using the E key. When turned off, all joints smoothly return to their initial positions when the `expositorTime` accumulator is reset:

```
if (keys[GLFW_KEY_E])
{
    animExpositor = !animExpositor;
    if (!animExpositor)
    {
        expositorRotMano = 0.0f;
        expositorRotCabeza = 0.0f;
        expositorTiempo = 0.0f;
    }
}
```

Robotic Arm Animation

Objective

An articulated robotic arm is positioned on the table at one of the booths. The animation reproduces the complete cycle of industrial manipulation: the arm picks up an imaginary part, lifts it, rotates it to the opposite side, sets it down, and returns to its resting position. The technique used is hierarchical keyframes, which combines keyframe animation with a parent-child chained transformation structure.

Data storage structure

Each keyframe stores the rotations of the arm's four joints, along with their respective increments per step and the total number of steps until the next keyframe:

```
typedef struct _brazo_frame {
    float baseRot;      float baseRotInc;
    float hombroRot;    float hombroRotInc;
    float codoRot;      float codoRotInc;
    float garraRot;     float garraRotInc;
    int maxSteps;
} BRAZO_FRAME;
```

Cinematics hierarchy

The arm consists of five elements arranged in a parent-child hierarchy:

fixed base → rotating pedestal → shoulder → arm/elbow → gripper

The pivot rotation technique is implemented by composing three transformations: translation to the pivot point, rotation, and inverse translation to return to the original coordinate system:

```
glm::mat4 mPedestal = modelBrazo;
mPedestal = glm::translate(mPedestal, pivotePedestal);
mPedestal = glm::rotate(mPedestal, glm::radians(brazo_baseRot), glm::vec3(0.0f, 1.0f, 0.0f));
mPedestal = glm::translate(mPedestal, -pivotePedestal);
```

The pivots for each joint (base, shoulder, elbow, and claw) were extracted directly from the model in Blender

Defining the animation cycle

Nine keyframes were defined (ARM_MAX_FRAMES = 9) to describe the complete manipulation cycle:

```
BrazoKF[0] = { 0.0f, 0, 0.0f, 0, 0.0f, 0, 0.0f, 0, 100 }; // reposo
BrazoKF[1] = { 45.0f, 0, 0.0f, 0, 0.0f, 0, 0.0f, 0, 120 }; // gira base
BrazoKF[2] = { 45.0f, 0, 30.0f, 0, 10.0f, 0, 5.0f, 0, 150 }; // baja
BrazoKF[3] = { 45.0f, 0, 30.0f, 0, 10.0f, 0, -5.0f, 0, 60 }; // balanceo
BrazoKF[4] = { 45.0f, 0, -10.0f, 0, -30.0f, 0, 0.0f, 0, 120 }; // levanta
BrazoKF[5] = { -45.0f, 0, -10.0f, 0, -30.0f, 0, 5.0f, 0, 180 }; // gira
BrazoKF[6] = { -45.0f, 0, 28.0f, 0, 8.0f, 0, 5.0f, 0, 150 }; // baja
BrazoKF[7] = { -45.0f, 0, 0.0f, 0, 0.0f, 0, 0.0f, 0, 120 };
BrazoKF[8] = { 0.0f, 0, 0.0f, 0, 0.0f, 0, 0.0f, 0, 1 };
```

Animation Playback

During each iteration of the game loop, joint rotations are updated by adding the increments previously calculated by the `brazoInterpolation()` function. When the maximum number of steps for the current keyframe is reached, the system moves on to the next keyframe and recalculates the increments:

```

if (brazoPlay) {
    if (brazo_i_curr_steps >= brazo_i_max_steps) {
        brazoPlayIndex++;
        if (brazoPlayIndex >= BRAZO_MAX_FRAMES - 1) {
            brazoPlayIndex = 0;
            brazo_i_curr_steps = 0;
            brazoResetElements();
            brazoInterpolation();
        }
        else {
            brazo_i_curr_steps = 0;
            brazoInterpolation();
        }
    }
    else {
        brazo_baseRot += BrazoKF[brazoPlayIndex].baseRotInc;
        brazo_hombroRot += BrazoKF[brazoPlayIndex].hombroRotInc;
        brazo_codoRot += BrazoKF[brazoPlayIndex].codoRotInc;
        brazo_garraRot += BrazoKF[brazoPlayIndex].garraRotInc;
        brazo_i_curr_steps++;
    }
}
}

```

Playback Controls

The animation starts automatically when the program is launched, plays on an infinite loop, and can be paused or resumed using the B key. When paused, the arm retains the exact position of the current frame; when resumed, it continues from that point without any visual jumps

Robot Dog Animation

Objective

The CROFI booth features a robotic dog model animated using skeletal animation. The robotic dog was created entirely by hand: the rigging was built vertex by vertex in Blender, and the animation was generated using keyframes on the software's timeline, allowing for the design of movement specific to the booth's context.

Technical Basis

Skeletal animation allows the mesh of a three-dimensional model to be deformed based on the transformations of an internal skeleton (bones), where each vertex is associated with one or more bones via influence weights. The final position of each vertex during rendering is calculated as a weighted sum of the transformations of the bones affecting it:

$$\text{final_position} = \sum (\text{weight}_i \cdot \text{bone}_i\text{-matrix} \cdot \text{local_position})$$

where the sum is taken over all bones i that influence the vertex; $weight_i$ is the influence factor of each bone; $bone_i_matrix$ is the cumulative transformation of the bone at the current time step; and $local_position$ is the original coordinate of the vertex in the mesh's rest reference frame

Implementation Process

The complete implementation process was carried out in five stages:

Modeling and rigging in Blender: The robot dog's skeleton was built manually, defining bones for the four legs, the hip, the torso, and the head.

Weight assignment: Each vertex of the mesh was linked to one or more bones with its respective influence weight, manually adjusting the transition zones between bones to avoid abrupt deformations.

Frame-by-frame animation creation in Blender: A greeting animation was generated by rotating the bones on the timeline.

Exporting to FBX format: the final file includes the mesh, skeleton, weights, and baked animation.

Loading into OpenGL via Assimp: the `ModeloAnimado` class processes the bone hierarchy, animation channels, and per-vertex weights, and sends them to the `anim.vs` shader for GPU processing.

Animation playback

The animation is controlled by a timer and a status flag. When the `playPerro` flag is active, the timer advances to the current frame time and is passed as a parameter to the model's `Draw()` method:

```
if (playPerro) {
    tiempoPerro += deltaTime;
}
perroRobot.Draw(shaderAnimacion, tiempoPerro);
```

Inside, the `ModeloAnimado` class uses the received time to interpolate between the keyframes of the animation loaded from the FBX, calculating the transformation matrix for each bone at the exact moment and sending it to the vertex shader as part of the bone uniform array

Playback Control

By default, the animation starts paused. When you press the P key for the first time, the dog begins to move; when you press it again, it stops and returns to frame zero of the cycle:

```
if (key == GLFW_KEY_P && action == GLFW_PRESS) {
    playPerro = !playPerro;
    if (!playPerro) tiempoPerro = 0.0f;
}
```

8. Cost Analysis

This analysis quantifies the financial resources invested in developing the individual component of the interactive virtual environment, as well as the estimated selling price of my contribution if it were offered as a professional service to an external client. All financial data is supported by verifiable sources from the Mexican market consulted in May 2026, which are listed in the bibliography section.

8.1 Development cost

My role in the project is that of a junior computer graphics developer, a position that combines C++ programming with OpenGL, 3D modeling in Blender, shader management, and animation. The rate is set at \$200.00 MXN/hr, a figure that falls within the lower range reported by the Mexican industry for junior developers (between \$150 and \$300 Mexican pesos per hour for junior developers in Mexico), considering that the 3D graphics development profile requires specialized knowledge beyond that of a generic junior developer.

The estimated hours are calculated based on my commit history in the GitHub repository and the actual duration of the project (from April 19 to May 26, 2026, equivalent to 5.5 weeks). An effective workweek of approximately 22 hours is assumed, which accounts not only for net coding time but also for hours spent on iterative testing, debugging, texture adjustments, pivot calibration in Blender, and visual validation of the animations.

Technical scope	Hours per week	Weeks	Total hours	Rate (MXN/hr)	Subtotal (MXN)
Bridge modeling, skybox system with day/night cycle, grandstand lighting (24 spotlights), and four animations (visitor, presenter, robotic arm, brochures)	22	5.5	120	200	24000
Total labor			120		24,000

8.2 Software cost

The following table lists each tool used, along with its corresponding license and its equivalent retail price for reference.

Software / Tools	Version	License type	Actual cost (MXN)	Equivalent retail cost (MXN)
Visual Studio Community 2022	17.x	Free for students and personal projects	\$0.00	\$3,480.00 (Professional)
Blender	4.x	GPL (Open Source)	\$0.00	\$0.00
OpenGL 3.3 Core Profile	3.3	Khronos Group open specification	\$0.00	\$0.00
GLEW (OpenGL Extension Wrangler)	2.1.0	Modified BSD / MIT	\$0.00	\$0.00
GLFW (Graphics Library Framework)	3.3	zlib/libpng	\$0.00	\$0.00
GLM (OpenGL Mathematics)	0.9.9	MIT	\$0.00	\$0.00
SOIL2 (Simple OpenGL Image Library)	1.x	MIT-0	\$0.00	\$0.00
Assimp (Open Asset Import Library)	5.x	BSD 3-Clause	\$0.00	\$0.00
Mixamo (skeleton animations)	Web	Free with an Adobe account	\$0.00	\$0.00
Git / GitHub	2.x	Free for public repositories	\$0.00	\$0.00
Total software (actual cost)			\$0.00	\$3,480.00

8.3 Hardware cost

For the development and testing of my part of the project, I used a personal computer. The reference price corresponds to a mid-range gaming laptop with a dedicated GPU, positioned in the mid-range segment of the Mexican market (gaming laptops in Mexico with an RTX 4050 or higher range from \$22,000 to \$36,000 MXN). Since this computer performs additional functions outside the project, its cost is amortized over a 5-year useful life, and only the portion corresponding to the development time (5.5 weeks out of a total of 260 weeks) is allocated.

Item	Quantity	Unit cost (MXN)	Useful life (weeks)	Weeks used	Allocable cost (MXN)
Laptop with a dedicated GPU (mid-range))	1	25000	260	5.5	528.85
External devices (mouse, keyboard, external monitor)	1 set	3500	260	5.5	74.04
Total hardware					602.89

8.4 Indirect costs

Residential internet rates are based on the basic Telmex Infinitum plan (MXN 349 per month for 80 Mbps, increasing to MXN 449 for 150 Mbps), with an average of MXN 399 per month. Electricity consumption is calculated based on the average CFE residential rate for a laptop with a 150W charger operating throughout the project.

Item	Description	Cost (MXN)
Internet service	\$399.00/month x 1.4 months (5.5 weeks)	558.6
Electricity	Use of computer equipment for 120 hours of actual usage	350
Office supplies	Prints, notes, and sketches on paper	75
Total indirect costs		983.6

8.5 Cost Summary

Category	Cost (MXN)	Per cent
Labor	24000	93.50%
Software	0	0.00%
Hardware	602.89	2.35%
Indirect costs	983.6	3.83%
Subtotal: Development cost	25586.49	99.68%
Contingency (0.32%)	82.42	0.32%
TOTAL DEVELOPMENT COST	25,668.91	100.00%

Project sale price

The selling price is calculated based on the total development cost, plus a reasonable profit margin, and taking into account the administrative costs associated with delivering the product as a professional service. This price reflects the amount an external client (such as an educational institution, an event sponsor, or a student organization) would pay to receive the portion of the interactive virtual environment corresponding to my contribution.

Item	Estimation	Amount (MXN)
Development cost	Total for the Cost Summary section	25668.91
Administrative expenses	Billing, project management, client communication	2566.89
Post-delivery technical support (3 months, 8%)	Bug fixes and minor adjustments	2053.51
Subtotal		30289.31
Profit margin (40%)	Net profit from development, standard for the specialized graphic design industry	12115.72
Net price before tax		42405.03
VAT (16%)	Value-added tax in Mexico	6784.8
FINAL SELLING PRICE		49,189.83

Justification of the price

The final selling price of \$49,189.83 MXN is considered reasonable within the Mexican market for the development of interactive 3D graphics applications. To gauge its value, four comparable market alternatives across different levels of complexity are compared below:

Market Alternative	Approximate cost (MXN)	Notes
360° virtual tour for website (static photography)	\$1,500 - \$7,000	Navigation between panoramic photos only; no interactivity or animations
Residential virtual tour with VR	~\$25,000	Immersive experience but no custom interactive logic
Highly complex residential virtual tour	\$2,000 - \$185,000	Wide range depending on the level of detail and animation
Interactive application in Unity using custom assets	\$50,000 - \$120,000	Native 3D app with interactivity and complex animations
My contribution to the project (native OpenGL)	\$49,189.83	Native 3D app, 4 distinct animations, dual skybox system, custom modeling

Note: This cost analysis pertains exclusively to my individual contribution to the collaborative project, not to the entire project developed by the team. The figures presented here reflect only the investment, resources, and commercial value associated with the deliverables for which I was responsible. The portion corresponding to the other team member is not included in this document.

9. Conclusions

The development of this project presented a considerable technical and managerial challenge, but it also offered a real opportunity to apply all the concepts covered in the course in an integrated manner. The construction of an interactive virtual environment that faithfully reproduces the bridge connecting Buildings A and B of the School of Engineering, along with the implementation of the Phong lighting model, the dual skybox system, and the seven independent animations, demonstrated that it is possible to combine theory and practice into a functional and visually coherent product.

My individual contribution to the project focused on the 3D modeling of the bridge, its texturing, the skybox system with dynamic day/night cycling, the twenty-four spotlight-type lights on the stands, and four of the seven environment animations: the walking visitor, the articulated presenter, the hierarchical robotic arm, and the flying brochures; each of these tasks represented a distinct technical learning experience.

Beyond the technical aspects, the project reinforced the importance of professional teamwork management through the use of GitHub.

I believe the project achieves its academic objectives and demonstrates a comprehensive mastery of the course concepts. As a practical application, the virtual environment developed functions as a prototype for a virtual tour that could be used to plan the layout of booths at real university events, show corporate sponsors what their presence on campus would look like before a physical event, and reduce the logistical costs associated with in-person setups. The learning curve was steep, but the final result fully justifies the effort invested.

10. Bibliography

Blender Foundation. (s.f.). 5.1 — Blender. Blender.
<https://www.blender.org/download/releases/5-1/>

How to Use Mixamo with Blender: Full Beginner Guide. (s.f.). RebusFarm.
<https://rebusfarm.net/blog/how-to-use-mixamo-with-blender-full-beginner-guide>

Modelos descargados de internet Render People 3D models. (04 de enero de 2022). 3DTEAMZ. <https://3dteamz.com/render-people-3d-models/>

Industrial Robot Brazo Gratis 3D Modelos Stl - .stl descargar - Free3D. (s.f.). https://free3d.com/es/modelos-3d/stl-industrial-robot-brazo?dd_referrer=

Moon with 2k Textures | 3D model. (15 de diciembre de 2023). CGTrader. <https://www.cgtrader.com/free-3d-models/space/planet/moon-with-2k-textures>

Robot Dog | 3D model. (05 de mayo de 2023). CGTrader. <https://www.cgtrader.com/free-3d-models/science/other/robot-dog-5cf6776b-9095-4e6b-8c79-26f2a05ef202>

Video camera | CGTrader. (s.f.). CGTrader. <https://www.cgtrader.com/items/6117823/download-page>

Meshy. (13 de mayo de 2026). “Black Dome Pendant Light” - Descarga modelos 3D gratuitos generadas con Meshy AI por @dr.kanguro10. Meshy AI - El Generador De Modelos 3D Con IA Número 1. https://www.meshy.ai/es/3d-models/Black-Dome-Pendant-Light-v2-019e19eb-e4c7-7755-944d-51037603dd43?utm_medium=referral-program&utm_source=meshy&utm_content=T2AFGA&share_type=3d-models

Industrial Table Bar 120x30x112 | 3D model. (s.f.). CGTrader. <https://www.cgtrader.com/free-3d-models/interior/office-interior/industrial-table-bar-120x30x112-9d02243d-fe1c-46b6-864f-f602f1e0ca68>

Computrabajo México. (2026). Salario de Becario/a en México 2026. mx.computrabajo.com/salarios/becario

Glassdoor. (2026, marzo). Sueldo: Becario en desarrollo de software en México. glassdoor.com.mx/Sueldos/becario-en-desarrollo-de-software-sueldo-SRCH_KO0,33.htm

Accademia. (2024, 21 de diciembre). Costo por hora de desarrollo de software en México. accademia.es/costo-por-hora-de-desarrollo-de-software-en-mexico/

UNIR México. (2026, 13 de marzo). Desarrollador de software, qué hace y cuánto gana. mexico.unir.net/noticias/ingenieria/desarrollador-software/

Hardysystem. (2026). Visual Studio Professional 2026: IDE avanzado para desarrollo en Windows. hardysystem.com/productos/visual-studio-professional-2026-jpd81/

Microsoft Corporation. (2026). Comprar y descargar Visual Studio Professional 2026: Microsoft Store. microsoft.com/es-mx/d/visual-studio-professional-2026/dg7gmgf0vj96

CityShop. (2026, 11 de marzo). PC de Escritorio vs Laptop en 2026: Guía de compra. cityshop.mx/blog/pc-escritorio-vs-laptop-2026-guia-compra

Intercompras. (2026, 21 de abril). ¿Cuál es la mejor laptop gamer barata? intercompras.com/blog/mejor-laptop-gamer-barata/

Ámbito Financiero. (2026, 29 de enero). Cuál es internet más barato que conviene contratar en febrero 2026 en CDMX: precio de las tarifas de CFE, Starlink y Telmex. ambito.com/mexico/economia/cual-es-internet-mas-barato-que-conviene-contratar-febrero-2026-cdmx-precio-las-tarifas-cfe-starlink-y-telmex-n6239953

El Informador. (2026, 17 de marzo). Telmex, Izzi y CFE: ¿Cuál es el proveedor de internet más barato en México en 2026? informador.mx/mexico/telmex-izzi-y-cfe-cual-es-el-proveedor-de-internet-mas-barato-en-mexico-en-2026-20260317-0102.html

Habitissimo México. (2019, 11 de diciembre). Hacer recorrido virtual: Precio y cotizaciones. habitissimo.com.mx/presupuesto/hacer-recorrido-virtual

Cronoshare México. (2024, 3 de mayo). ¿Cuánto cuesta un Tour Virtual 360? Precios en 2026. cronoshare.com.mx/cuanto-cuesta/tour-virtual-360

Aulacm. (2026, 31 de enero). Cuánto cuesta una App móvil y cómo desarrollarla. Precio y tipos. aulacm.com/precio-desarrollar-app-aplicacion-movil/